Frozen Lake from First Principles using Q-Learning

DSCD 614 – Reinforcement Learning · Programming Assignment 1
University of Ghana · Department of Computer Science

Name	Theophilus Tetteh Ayernor	Student ID	22424572
GitHub Repository	https://github.com/TheoTheo-rgb/Reinforcement-Learning-Project-Work		
Report Link	https://drive.google.com/file/d/1lhKMP9pv2jh8KymGtPZusVeT2zCcwk_G/view?usp=sharing		

1. Introduction

Reinforcement Learning (RL) is a paradigm in which an agent learns to act by interacting with an environment. At each step the agent observes a state, chooses an action, and receives a scalar reward and the next state. It is never told the correct move; instead it must discover, by trial and error, a policy (state→action mapping) that maximises the expected sum of discounted future rewards, continually balancing exploration of new actions against exploitation of good ones.

Frozen Lake is a classic grid-world benchmark. An agent must cross a frozen lake from a Start cell (S) to a Goal cell (G), stepping only on Frozen tiles (F) and avoiding Holes (H), which end the episode in failure. This work solves the standard 8×8 map entirely from first principles — the environment, agent, training loop and evaluation are written in pure Python/NumPy with no RL frameworks (Gymnasium, Gym, Stable Baselines, RLlib).

2. Environment Design

The environment is the `FrozenLakeEnv` class, exposing `reset()`, `step(action)`, `render()`, `get_state()` and `is_terminal()`.

State representation. Each cell is encoded as a single integer index `state = row × n_cols + col` (0–63 for the 8×8 grid), with helpers converting to/from (row, col).

Action representation. Four actions: 0 = Left, 1 = Down, 2 = Right, 3 = Up. Moves that would leave the grid are clipped, keeping the agent in place (boundaries are enforced).

Reward structure. The classic sparse reward is used: +1.0 for reaching the Goal, 0.0 for falling into a Hole (terminal failure) and 0.0 for an ordinary step. Because the discount factor $\gamma < 1$, an earlier +1 is worth more than a later one, so the sparse reward already favours the shortest safe path without any reward shaping. The episode ends on reaching the goal, falling in a hole, or hitting a 200-step limit. An optional slippery mode adds stochastic transitions (bonus task).

3. Q-Learning Algorithm

Q-Learning is a model-free, off-policy, temporal-difference control method. It learns an action-value table Q(s, a) — the expected discounted return of taking action a in state s and acting greedily afterwards — initialised to zeros. After every transition (s, a, r, s′) it applies exactly:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$$

Here α (learning rate) controls how fast new experience overwrites old estimates; γ (discount factor) weights future versus immediate reward; and $r + \gamma \cdot \max Q(s',a')$ is the one-step bootstrap target. The bracketed quantity is the temporal-difference error; for a terminal s′ the bootstrap term is dropped. The optimal policy is read off as $\pi(s) = \arg\max_a Q(s, a)$.

Exploration strategy. Actions follow an ε-greedy rule: a random action with probability ε, otherwise the greedy action. ε starts at 1.0 and is decayed exponentially after each episode toward a floor of 0.01 ($\varepsilon \leftarrow \max(\varepsilon_{min}, \varepsilon \cdot \varepsilon_{decay})$), shifting the agent from exploration to exploitation as learning proceeds.

4. Training Methodology

Each episode resets the agent to the start and repeatedly selects an ε-greedy action, steps the environment, and applies the Q-update until termination, after which ε is decayed. Per-episode reward, success flag, step count and ε are logged. The default configuration, selected from the study in Section 5,

is:

Episodes	α (learning rate)	γ (discount)	ε schedule	ε decay	Max steps	Seed
20,000	0.1	0.99	1.0 → 0.01	0.9995	200	42

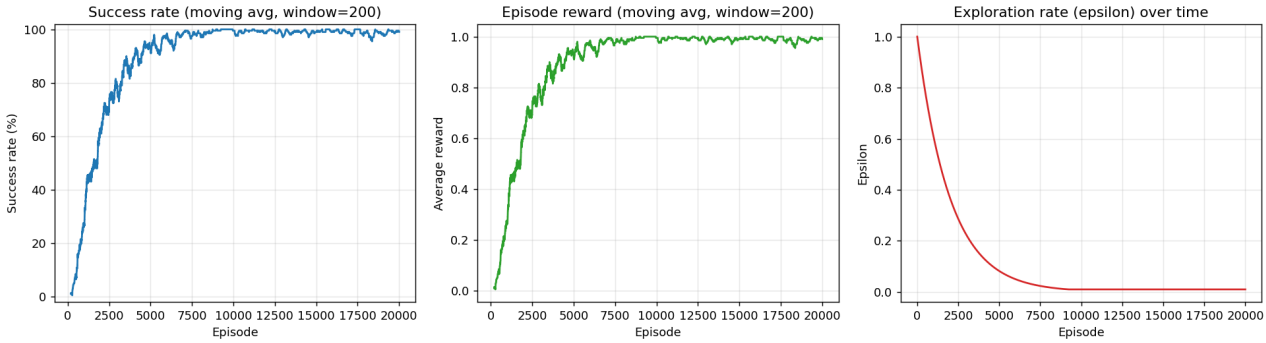

Figure 1. Training curves: success rate (left) and average reward (centre) rise to ~100% as ε (right) decays.

5. Experimental Results

Greedy evaluation of the final agent over 200 episodes on the deterministic map achieved a 100% success rate, an average reward of 1.000, 0 failures, and a 14-step path to the goal — equal to the Manhattan distance from S(0,0) to G(7,7), hence provably optimal.

Hyperparameter study. Learning rate and discount were each varied while training for 15,000 episodes. Every setting solved the task (100% greedy success); the meaningful difference is convergence speed, measured as the episode at which the 200-episode moving-average success first reaches 90%:

α (γ=0.99)	0.01	0.05	0.1	0.5
Episodes to 90%	3,619	4,016	3,545	4,101
Final success	100%	100%	100%	100%

A moderate α ≈ 0.1 learns fastest; very small or very large rates are slightly slower. Varying γ over {0.90, 0.95, 0.99} produced an identical optimal policy and convergence point (~3,545 episodes): for a single-goal deterministic shortest-path task γ rescales values but does not change the arg-max, so the policy is unchanged.

Exploration comparison (bonus). A decaying ε-greedy policy (1.0→0.01) was compared with a pure ε-greedy policy (fixed ε = 0.1). Both learn the same optimal greedy policy, but their training behaviour differs (Figure 2): fixed ε rises quickly yet plateaus near 93% because 10% of its actions stay random, whereas decaying ε starts slower but climbs to ~100% as exploration vanishes.

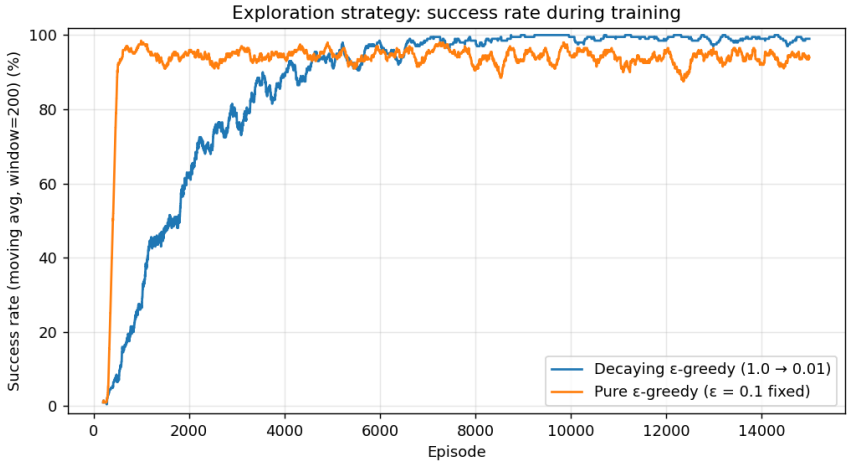

Figure 2. Pure ε-greedy (fixed ε) converges fast but caps below 100%; decaying ε-greedy reaches ~100% behaviour success.

Stochastic transitions (bonus). On the slippery variant — where the intended move occurs with probability 1/3 and the agent slips perpendicular with probability 1/3 each — the same algorithm (γ = 0.95) reached about 87% success over 1,000 greedy episodes, versus 100% deterministic (Figure 3). The remaining failures are intrinsic: random slips can force even an optimal policy into a hole, and the curve is far noisier.

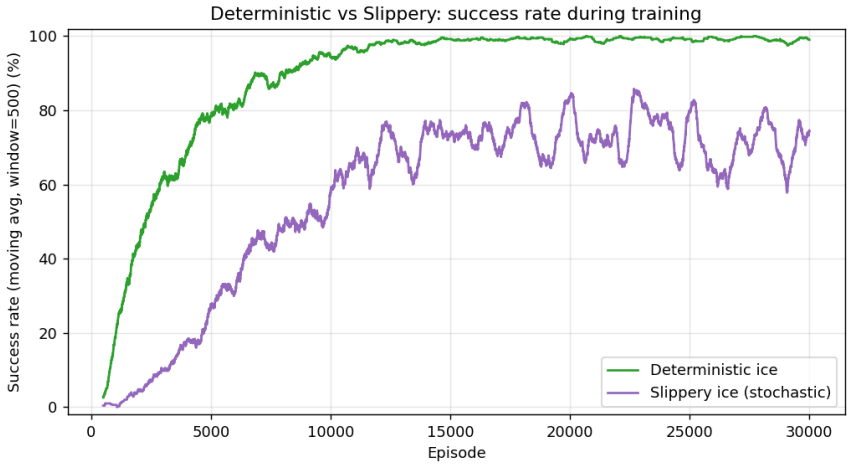

Figure 3. Deterministic dynamics converge to 100%; slippery dynamics plateau lower with much higher variance.

6. Learned Policy

Figure 4 shows the greedy action in every non-terminal cell (arrows), the state value $V(s) = \max_a Q(s, a)$ as a colour gradient, and the greedy trajectory (red). The agent moves down out of the start, right across row 1 to skirt the wall of holes in column 3, then down the right-hand columns to the goal in 14 steps. Values increase smoothly toward the goal, confirming correct value propagation; cells far off the optimal path retain near-zero value and arbitrary arrows because they are rarely visited.

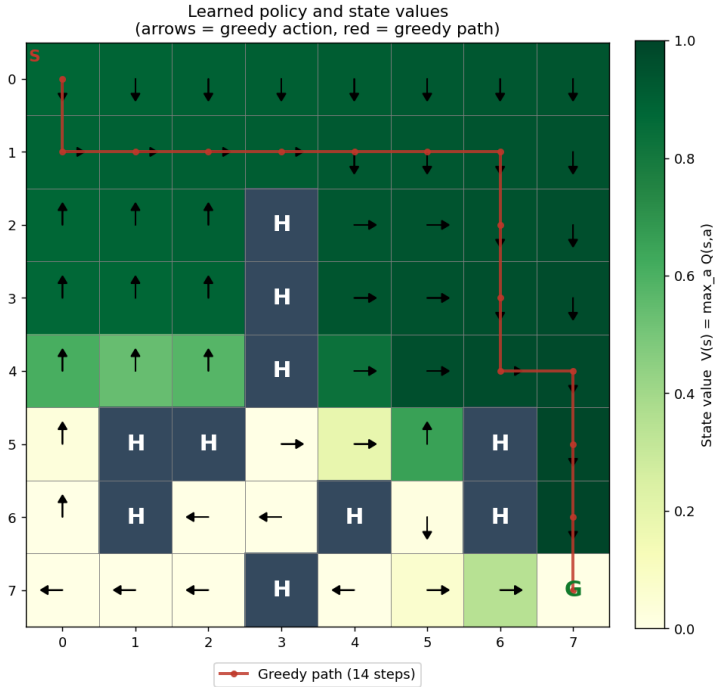

Figure 4. Learned policy and state-value heatmap with the 14-step optimal path traced in red.

7. Challenges Encountered

Sparse rewards. With reward only at the goal, the agent receives no signal until a random walk first reaches G; high initial ε and enough episodes were needed to obtain that first signal and propagate it backward. Exploration–exploitation balance. Too little exploration risked a sub-optimal route; too much

slowed convergence — tuning the ε decay schedule resolved this. The column-3 hole wall makes naïve shortest-path heuristics fail, so the agent had to learn the detour. Stochasticity. Slippery dynamics greatly increased variance and lowered achievable success, requiring more episodes and a slightly lower γ for stable behaviour.

8. Conclusion

A complete tabular Q-Learning solution to the 8×8 Frozen Lake was implemented from scratch without any RL framework. On the deterministic lake it learns a provably optimal 14-step policy with a 100% greedy success rate, and the learning curves clearly show convergence as exploration decays. The hyperparameter study confirms the solution is robust across learning rates and discount factors, and all three bonus tasks — stochastic transitions, performance visualisation, and an exploration-strategy comparison — were completed, with the slippery variant reaching ~87% success. The results demonstrate that the core RL machinery, when correctly implemented, reliably discovers optimal behaviour in this grid-world.